FitBuddy

Scalability and Future Plan

A staged path from local prototype to a more reliable multi-user service

Current starting point

The brief proposes FastAPI, Uvicorn, Gemini services, Jinja2, and SQLite with SQLAlchemy. This stack suits a learning project and local prototype. Measure actual usage and bottlenecks before changing the architecture.

Likely growth pressure points

AI calls: response time, rate limits, provider availability, and usage cost.
Database: concurrent writes, growing user history, and admin list queries.
Web app: slow requests if AI calls run synchronously in user-facing routes.
Operations: secrets, backups, access control, logs, and deployment monitoring.

Stage 1 - strengthen the prototype

Add input limits, explicit AI timeouts, helpful error handling, and structured logs.
Index user IDs and frequently filtered fields; paginate the admin results.
Keep API keys in environment/secret configuration; restrict admin routes.
Measure end-to-end, AI, and database latency separately.

Stage 2 - prepare for more users

Move to a managed relational database such as PostgreSQL when concurrent access, backup, or deployment needs exceed what local SQLite can support.
Run the app behind a production ASGI/web server setup; externalize persistent files.
Use a background job queue for longer AI generation if users need non-blocking flows.

FitBuddy

Roadmap and Readiness Checks

Stage 3 - improve product capability

Add account authentication and user-owned plan access.
Track plan revisions and user-approved progress measures.
Offer configurable goals, rest preferences, and accessibility options.
Add AI output checks and a clear way to report unsuitable suggestions.

Stage 4 - production operations

Automate deployment and database migrations; maintain tested backups.
Monitor availability, latency, error rates, resource use, and AI provider failures.
Set retention limits, data deletion procedures, and incident response steps.

When to scale - review evidence

Repeated slow responses under expected traffic; database locks or sustained high CPU.
Users waiting too long on AI calls; provider limits reached; backup/recovery gaps.
Choose the smallest change that addresses the measured bottleneck.

Success measures

Set target response times and error rates after a baseline and agree on expected load.
Track completion of plan generation, revision success, and user-reported usefulness.
Review costs per AI operation and resource use as adoption grows.

Guiding principle

Scale in response to evidence. Keep personal data private, preserve plan history,
and treat AI fitness content as general guidance rather than medical advice.